INMERSIÓN · INGENIERÍA CON AGENTES DE IA

Spec-Driven Development a fondo

Cómo diseñar, escribir y mantener especificaciones que dirigen a agentes de IA

SDD

spec.md · requirements.md · design.md · tasks.md

EL PROBLEMA

El agente no pregunta: rellena los huecos

Contexto que se pierde

Cada sesión empieza de cero y el agente olvida las decisiones previas.

Supuestos plausibles

Ante la ambigüedad, elige un valor por defecto y lo implementa con total confianza.

Cambios inconsistentes

Features que se contradicen porque nadie fijó el contrato entre ellas.

Revisión imposible

Miles de líneas generadas sin un criterio escrito contra el cual revisarlas.

SDD responde con un **contrato explícito** antes de que el agente empiece a escribir código.

DEFINICIÓN

La spec es la fuente de verdad; el código, un artefacto generado y verificable

Se escribe una especificación estructurada y orientada al comportamiento antes de que la IA genere código, y se conserva como contrato compartido entre personas y agentes.

01
Intención antes que implementación

02
Depurar la spec, no el código

03
Verificar contra la spec

DE DÓNDE VIENE

Una idea vieja con un ejecutor nuevo

Antecedentes

1986
Design by Contract (B. Meyer)

2001
Model-Driven Architecture (OMG)

2006
BDD y escenarios Gherkin

2009
Sintaxis EARS para requisitos

2010s
API-first con OpenAPI

Nacimiento del SDD con agentes

Feb 2025
Karpathy nombra el “vibe coding”

Jul 2025
AWS lanza Kiro

Sep 2025
GitHub Spec Kit y Tessl

Oct 2025
Böckeler define tres niveles

2026
Corriente principal: papers y 30+ frameworks

NO CONFUNDIR

Spec, prompt, PRD y memory bank

Artefacto	Vida útil	Alcance	Ejemplo
Prompt	Efímero: vive en el chat	Una petición	“Agrega login con Google”
PRD	Largo plazo, escrito para personas	Un producto completo	Documento de producto
Spec	Persistente, versionada y revisable	Una feature o un cambio	specs/login/spec.md
Memory bank	Persistente y transversal	Todas las sesiones del repo	constitution.md, steering, AGENTS.md

La spec describe **una funcionalidad**; el memory bank describe **el proyecto**. Ambos se versionan junto al código.

PARTE 02

Anatomía de una spec

Capas, sintaxis de requisitos y lo que distingue una spec útil de un documento que nadie lee.

LAS CAPAS

Cinco capas, de lo general a lo verificable

Memory bank	Principios y reglas que aplican a todo el proyecto	`constitution.md · steering/`
Requisitos	Qué y por qué: historias y criterios verificables	`requirements.md · spec.md`
Diseño	Cómo: arquitectura, interfaces, datos y errores	`design.md · plan.md`
Tareas	Pasos pequeños, ordenados y trazables a requisitos	`tasks.md`
Evidencia	Tests, checks y estado verificado de cada tarea	`CI · checkboxes · reportes`

Las herramientas cambian los nombres de archivo, pero casi todas reproducen esta estructura.

EARS · EASY APPROACH TO REQUIREMENTS SYNTAX

Requisitos sin ambigüedad

Patrón	Plantilla	Ejemplo
Ubicuo	El sistema DEBE…	El sistema DEBE registrar cada reembolso en la auditoría.
Por evento	CUANDO <disparador>, el sistema DEBE…	CUANDO el cliente pide un reembolso, el sistema DEBE confirmarlo por correo.
Por estado	MIENTRAS <estado>, el sistema DEBE…	MIENTRAS haya un reembolso en curso, el sistema DEBE bloquear otro igual.
No deseado	SI <condición>, ENTONCES el sistema DEBE…	SI el pago falla, ENTONCES el sistema DEBE reintentar una vez.
Opcional	DONDE <característica>, el sistema DEBE…	DONDE exista billetera, el sistema DEBE ofrecer abonar el saldo.

EJEMPLO

Un fragmento de requirements.md

```
## Requisito 1 · Reembolsos
Historia: como cliente quiero pedir
un reembolso sin llamar a soporte.
### Criterios de aceptación
1.1 CUANDO pida un reembolso de un pedido
    de menos de 30 días, el sistema
    DEBE aprobarlo automáticamente.
1.2 SI el pedido supera 30 días, ENTONCES
    el sistema DEBE derivarlo a revisión.
### Fuera de alcance
- Reembolsos parciales
```

La historia

Da el porqué: el agente decide mejor con contexto.

Criterios EARS

Cada uno se convierte en un test.

Numeración

Diseño y tareas citan “1.2”: hay trazabilidad.

Fuera de alcance

Lo que el agente no debe construir.

CRITERIOS DE CALIDAD

Qué hace buena a una spec

✓ Verificable
Cada criterio puede convertirse en un test.

✓ Acotada
Declara qué queda fuera de alcance.

✓ Trazable
Cada tarea cita el requisito que cumple.

✓ Separa qué y cómo
Requisitos sin detalles técnicos; el cómo va al diseño.

✓ Decisiones explícitas
Lo decidido y por qué, en ADRs o en el diseño.

✓ Proporcional
Tan larga como el cambio lo exige, no más.

PARTE 03

Nuestra herramienta: Spec Kit

Primero, Spec Kit a fondo. Después, el resto del ecosistema: qué hace distinto y qué ideas podemos traer a nuestro flujo.

Qué queda en el repo

```
mi-proyecto/
├─ .specify/
│  ├─ memory/constitution.md
│  ├─ templates/overrides/
│  └─ scripts/
├─ .claude/skills/
└─ specs/001-reembolsos/
   ├─ spec.md
   ├─ plan.md · research.md
   ├─ data-model.md
   ├─ contracts/
   └─ tasks.md
```

```
# instalar fijando la versión
uv tool install specify-cli --from
  git+https://github.com/github/spec-kit.git@v1.0.6
specify init . --integration claude
```

Capas de plantillas: gana la primera que exista

1	**Overrides del proyecto:** las nuestras
2	**Presets:** flujos como bug o assess
3	**Extensiones:** comandos y plantillas extra
4	**Core** de Spec Kit

SPEC KIT · LA CONSTITUCIÓN

Escribir principios que se puedan comprobar

```
# Constitución del proyecto
## I. Tests primero
Toda feature incluye tests que fallan
antes de implementarse.
## II. Acceso a datos
Solo a través del paquete data/.
Una tarea que lo viole se reescribe.
## III. Simplicidad
Ninguna dependencia nueva sin un ADR.
## Gobernanza
Cambiar un principio requiere un PR
aprobado por el tech lead.
```

Reglas, no aspiraciones

“Código limpio” no se puede comprobar; “solo vía data/” sí.

Pocas y estables

Se aplica a cada feature: cada palabra pesa en todas.

Respaldadas por CI

El modelo interpreta la constitución; el linter y los tests la hacen cumplir.

SPEC KIT · LÍMITES CONOCIDOS

Lo que hay que saber antes de escalarlo

Límite	Cómo lo mitigamos
“Falso terminado”: los documentos parecen completos y faltan código o tests	Tests y CI son la única definición de terminado
tasks.md enormes y difíciles de revisar	Features pequeñas; dividir las specs grandes
Features que cruzan varios repos, sin soporte	Una spec por repo, enlazadas entre sí
Monorepos sin herencia de constitución	Sincronizar las constituciones con un script
Modernizar un sistema legado entero excede el contexto	Specs acotadas a cada cambio
El modelo juzga si se cumple la constitución	Reglas críticas también en linters y CI

OTRAS HERRAMIENTAS · KIRO (AWS)

Tres archivos y una aprobación en cada fase

`requirements.md`
Historias y criterios en EARS

↓ aprobación humana

`design.md`
Arquitectura, interfaces, datos

↓ aprobación humana

`tasks.md`
Tareas con dependencias, en paralelo

Steering

El memory bank: product.md, structure.md y tech.md.

Hooks

Acciones del agente al crear, guardar o borrar archivos.

Dos modos

Vibe para lo pequeño, spec para lo grande; Quick Spec genera todo sin gates.

La verdad actual y los cambios, separados

`openspec/specs/`

Lo que el sistema hace hoy

`openspec/changes/`

Un folder por cambio: proposal, design, tasks y deltas

`propose`
Proponer

→

`apply`
Implementar

→

`archive`
Fusionar deltas

Fuerte en sistemas existentes: cada cambio declara solo lo que agrega, modifica o elimina, y varios cambios pueden avanzar en paralelo.

```
## ADDED Requirements
### Requirement: Doble factor
#### Scenario: código válido
- WHEN el usuario ingresa
  un código válido
- THEN se emite la sesión
## MODIFIED Requirements
...
## REMOVED Requirements
...
```

OTRAS HERRAMIENTAS · TESSL Y BMAD-METHOD

Los dos extremos: generar desde la spec y orquestar roles

Tessl

CLI + servidor MCP + registro

La apuesta más ambiciosa: apunta a spec-as-source. Una spec por archivo de código, y el código generado lleva una marca de "no editar".

```
// GENERATED FROM SPEC - DO NOT EDIT
```

Reto: la misma spec produjo código distinto en cada generación.

BMAD-METHOD

Agile con agentes especializados

Cada rol es un agente: el analista escribe el brief, el PM el PRD, el arquitecto el diseño, y los *story files* llevan todo el contexto al agente que implementa.

```
Analista → PM → Arquitecto → Dev
```

Escala: pistas según tamaño, de Quick Flow a Enterprise.

SÍNTESIS

Qué traemos de cada una a Spec Kit

Herramienta	Qué hace distinto	Qué adoptamos en Spec Kit
Kiro	IDE propio, aprobación obligatoria en cada fase y hooks por eventos	Criterios en EARS y aprobación explícita entre spec, plan y tareas
OpenSpec	Separa la verdad actual de los cambios, con specs delta	En código existente, specs acotadas que describen solo el cambio
Tessl	La spec es la fuente; el código generado no se edita	Marcar los archivos generados y no parchearlos a mano
BMAD	Un agente por rol, del analista al desarrollador	Tareas con contexto completo, listas para implementarse

Cómo: con overrides de plantillas en .specify/templates/overrides/ y reglas en la constitución, sin cambiar de herramienta.

PARTE 04

SDD en la práctica

El ciclo de trabajo, cuánta spec escribir, cómo empezar en un sistema existente y quién hace qué.

EL CICLO

Ocho pasos y tres puntos de aprobación

01
Intención
El problema y el resultado buscado

→

02 · gate
Spec
Requisitos y criterios aprobados

→

03
Clarificar
El agente pregunta lo ambiguo

→

04 · gate
Plan técnico
Diseño aprobado

05 · gate
Tareas
Pequeñas y trazables

→

06 · agente
Implementar
Tarea a tarea, con tests

→

07
Verificar
Contra la spec, no contra la intuición

→

08
Actualizar
La spec refleja lo construido

Recuadro con borde grueso = gate: una persona aprueba antes de avanzar.

TAMAÑO PROPORCIONAL

¿Cuánta spec necesito?

Tamaño del cambio	Cuánta spec	Ejemplo de modo
Typo o bug trivial	Ninguna: instrucción directa	Modo vibe de Kiro
Cambio acotado	Criterios de aceptación y tareas	Quick Spec de Kiro
Feature mediana	Spec-first completa: requisitos, diseño, tareas	Spec Kit, specs de Kiro
Cambio en sistema existente	Spec delta sobre la verdad actual	changes/ de OpenSpec
Épica o cambio transversal	Spec-anchored con ADRs y revisión de arquitectura	BMAD, ASDD
Sistema crítico o regulado	Trazabilidad requisito → test → código	Gates en CI

BROWNFIELD

SDD en un sistema que ya existe

El reto

No hay specs, el conocimiento está en el código y en la cabeza del equipo.

No intentes especificar todo el sistema de una vez.

1 **Empieza por el memory bank**
Arquitectura, convenciones y glosario del dominio.

2 **Specs delta por cambio**
Describe solo lo que agrega, modifica o elimina.

3 **Spec inversa de lo crítico**
Que el agente describa un módulo y el equipo lo valide.

4 **Crece por uso**
La cobertura de specs avanza con cada cambio, no con un proyecto aparte.

VERIFICACIÓN

De la spec al test, y de vuelta

Criterio 1.2
SI supera 30 días...

→

Test derivado
test_deriva_a_revision

→

Implementación
El agente hace pasar el test

→

Gate en CI
Sin verde, no hay merge

Detectar la deriva entre spec y código

/speckit.analyze
Coherencia entre spec, plan y tareas

openspec validate
Estructura de cambios y escenarios

Hooks de Kiro
Acciones automáticas al guardar

Con LLM dentro
Agregar evals (EDD) a la spec

ROLES

¿Quién escribe y quién aprueba?

Producto

Escribe el qué y el porqué. Aprueba requisitos y fuera de alcance.

Tech lead

Mantiene la constitución, aprueba el diseño y registra decisiones.

Desarrollo

Revisa tareas y tests, resuelve lo difícil y valida la evidencia.

Agentes

Redactan borradores, preguntan, implementan por tarea y reportan.

La revisión se desplaza: menos líneas de código, más intención y criterios de aceptación.

PARTE 05

Límites y adopción

Lo que dicen los críticos, los errores frecuentes y una ruta para empezar sin romper nada.

CRÍTICAS

Lo que todavía no resuelve

Un flujo para todo

El mismo proceso pesado para un bug y para una épica.

Sobrecarga de revisión

Mucho markdown verboso y repetitivo; a veces es más fácil revisar código.

Falsa sensación de control

Los agentes ignoran instrucciones o las aplican de más.

Ecos de MDD

Riesgo de sumar la rigidez de los modelos al no determinismo de los LLM.

Definición difusa

“Spec” usado como sinónimo de prompt detallado.

¿Waterfall 2.0?

La respuesta: specs pequeñas y ciclos de minutos, no de meses.

ANTIPATRONES

Seis errores y cómo corregirlos

Antipatrón	Síntoma	Corrección
Spec = prompt largo	Se escribe en el chat y se pierde	Versionarla en el repo y revisarla en PR
Spec abandonada	El código ya no coincide con la spec	Actualizarla en el mismo PR del cambio
Sobre-especificar	Documentos que nadie lee	Tamaño proporcional y fuera de alcance explícito
Mezclar qué y cómo	Requisitos llenos de detalles técnicos	Requisitos en EARS; el cómo, al diseño
Parchear lo generado	Arreglos manuales que la spec ignora	Corregir la spec y regenerar
Confiar sin verificar	“El agente dice que terminó”	Tests derivados y gates en CI

RUTA DE ADOPCIÓN · PROPUESTA

Cómo empezar en cuatro etapas

Semana 1

Memory bank

Escribir la constitución: arquitectura, convenciones, comandos de test y glosario.

Semanas 2 y 3

Piloto spec-first

Una feature mediana con requisitos EARS, diseño y tareas. Medir el retrabajo.

Mes 2

Plantillas y gates

Plantillas del equipo, revisión de specs en PR y tests derivados de los criterios.

Mes 3 en adelante

Spec-anchored

Specs vivas en los módulos críticos y control de la deriva en CI.

Qué medir

PRs reabiertos o revertidos · tiempo de revisión · retrabajo tras el merge · specs desactualizadas

EN UNA FRASE

Depura la spec, no el código.

SDD no es escribir más documentos: es decidir qué es correcto antes de que el agente empiece, y mantener esa decisión viva.

REFERENCIAS

Fuentes

- B. Böckeler, Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl, martinfowler.com (oct. 2025)
- D. B. Piskala, “Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants”, arXiv 2602.00180 (2026)
- GitHub Spec Kit · Augment: guía práctica de Spec Kit (2026) · Spec Kit Deep Dive (DEV)
- Kiro: documentación de specs · OpenSpec: glosario · OpenSpec explicado
- BMAD-METHOD (Augment) · InfoWorld: 4 herramientas de SDD (2026)
- Augment: What Is Spec-Driven Development? · Developer Toolkit: fechas de lanzamiento
- A. Mavin y otros, “Easy Approach to Requirements Syntax (EARS)”, IEEE RE (2009)
- F. Zaninotto, “Spec-Driven Development: The Waterfall Strikes Back”, Marmelab